Seminarium Dyplomowe Inżynierskie - Część II

Rozwój portalu wspierającego obsługę rejsów dla biura Armatora
Wydział Oceanografii i Geografii UG

- Bartłomiej Krawisz 193319
- Paweł Pstrągowski 193473
- Stanisław Nieradko 193044

Agenda

1. Harmonogram prac
2. Stan pracy
3. Przegląd procesu przepisywania frontendu aplikacji
4. Porównanie narzędzi do testów automatycznych
5. Wybrane szczegóły implementacyjne w aplikacji

Harmonogram prac

Harmonogram prac

Termin	Zadanie
13 maja 2025	Zakończenie prac nad refaktorem aplikacji, wypuszczenie nowej wersji 2.0.0
czerwiec 2025 - październik 2025	Implementacja nowych funkcjonalności oraz naprawa znajdywanych błędów
21 września 2025	Wprowadzenie testów automatycznych do projektu
październik 2025 - grudzień 2025	Zapoznanie kolejnego zespołu z projektem
październik 2025 - grudzień 2025	Opis prac związanych z projektem w postaci pracy inżynierskiej
listopad 2025 - grudzień 2025	Wdrożenie nowej wersji aplikacji oraz ew. naprawa krytycznych błędów

Równolegle zaplanowane są regularne konsulatcje z promotorem oraz klientami w interwałach odpowiednio 2 tygodni oraz miesiąca.

Stan pracy

Stan pracy na moment pierwszej prezentacji

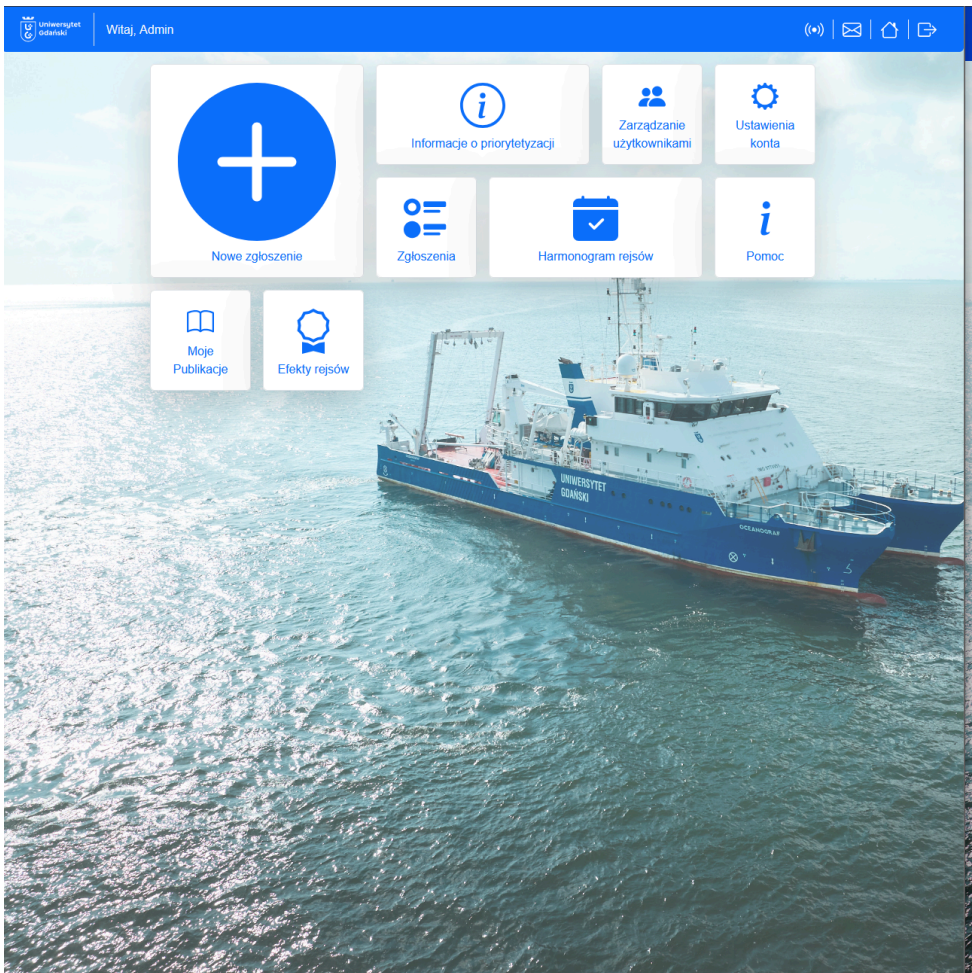
- Większość prac związanych z nowymi funkcjonalnościami dobiegła końca.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Aktualnie zajmujemy się:
  - Wdrażaniem nowej wersji na środowisko klienta
  - Wprowadzaniem nowego zespołu, który ma przejąć obowiązek utrzymywania aplikacji
  - Opisywaniem naszego projektu w pracy inżynierskiej.

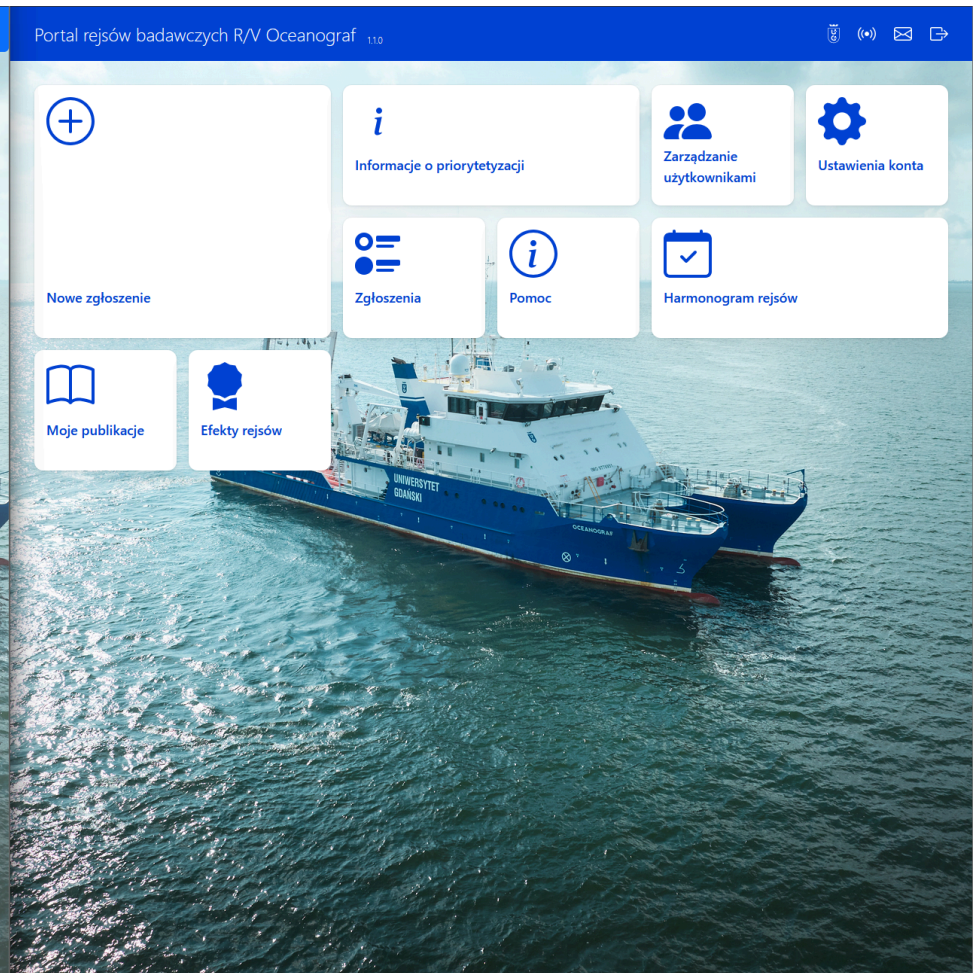
Aktualny stan pracy

- Nasz zespół zakończył już pracę nad dodawaniem nowych funkcjonalności.
- Testy automatyczne zgodnie z planem zostały wdrożone do projektu oraz pipeline'u CI.
- Nowa wersja aplikacji została wdrożona na środowisko klienta 20 listopada 2025.
- Nowy zespół został wdrożony w projekt, pracują oni aktualnie nad zgłoszonymi problemami i kolejnymi funkcjonalnościami.
- Większość naszej pracy inżynierskiej została już napisana, pozostały tylko końcowe poprawki.

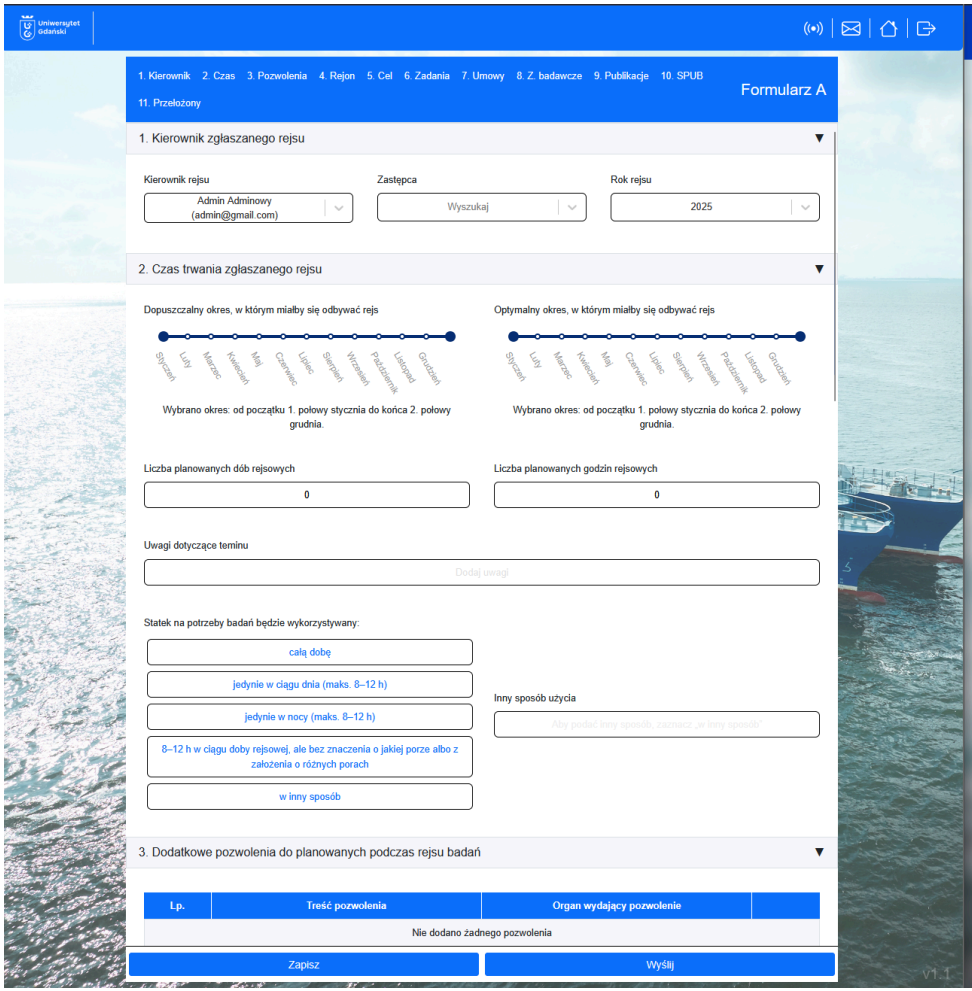
Przegląd procesu przepisywania frontendu aplikacji

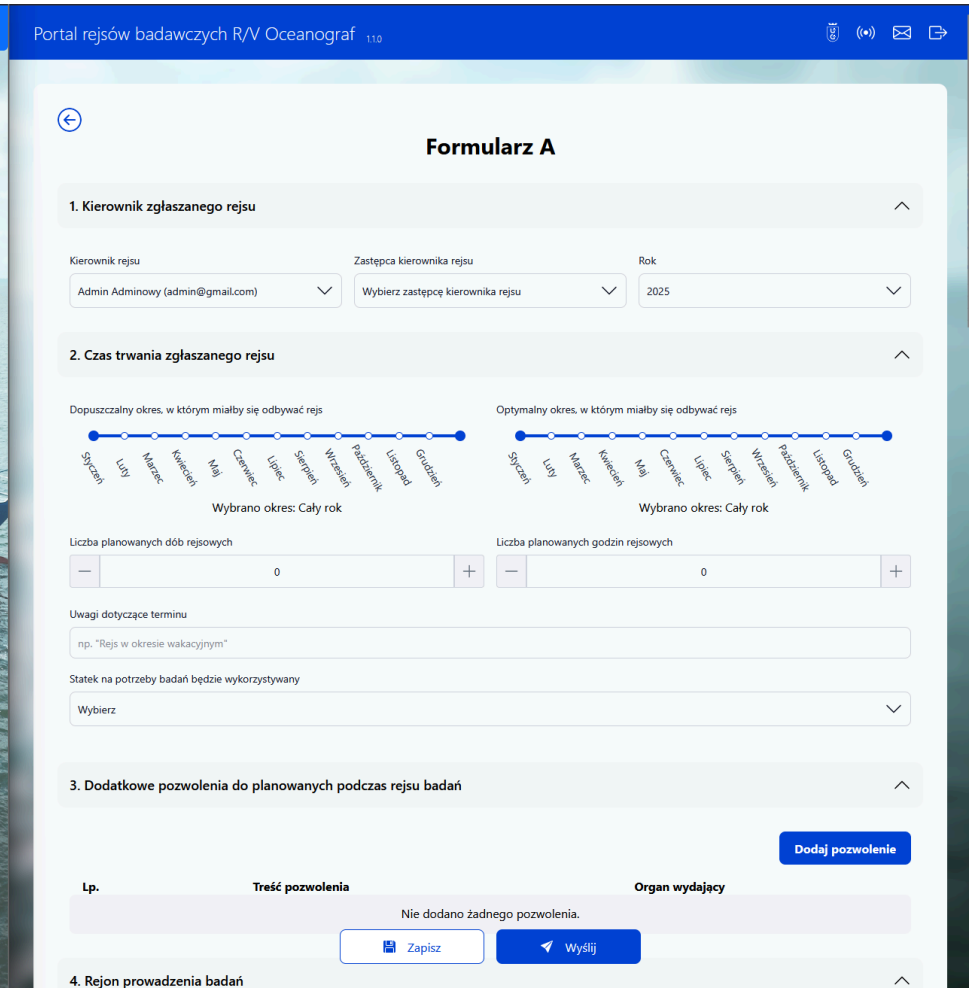
Porównajmy ponownie UX/UI przed i po



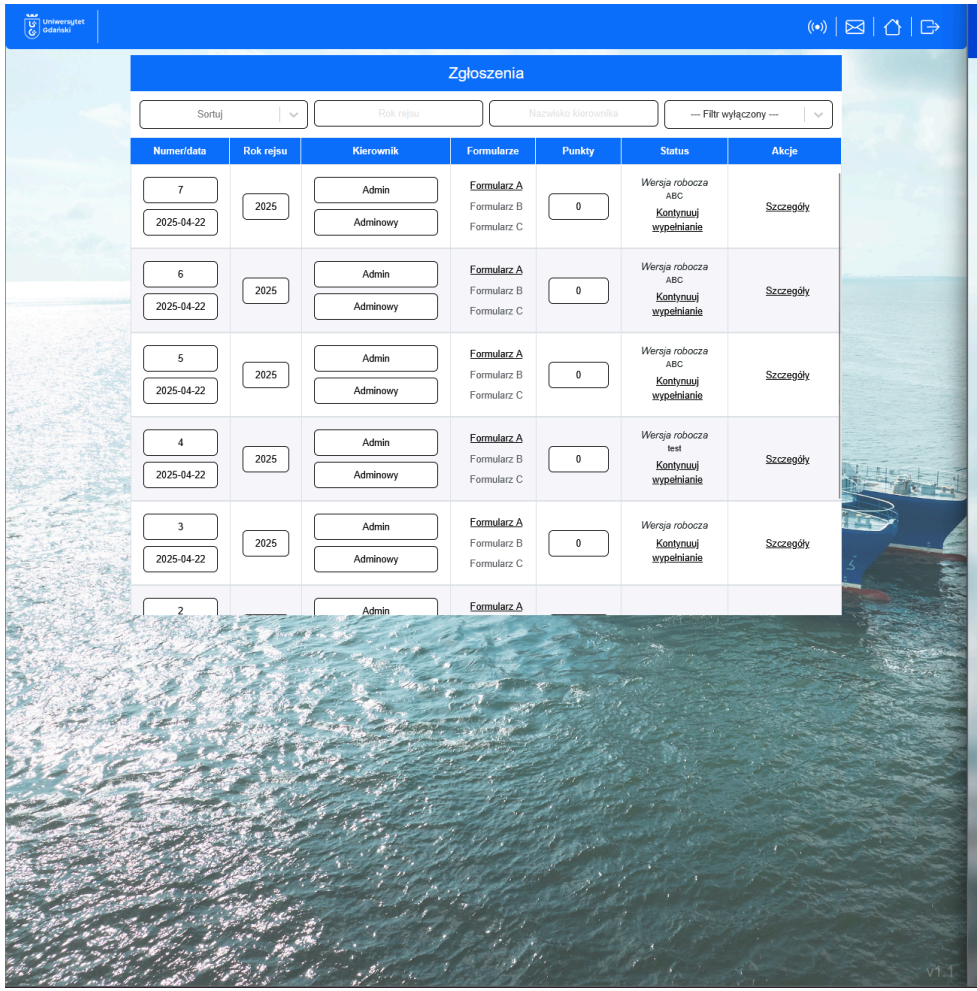


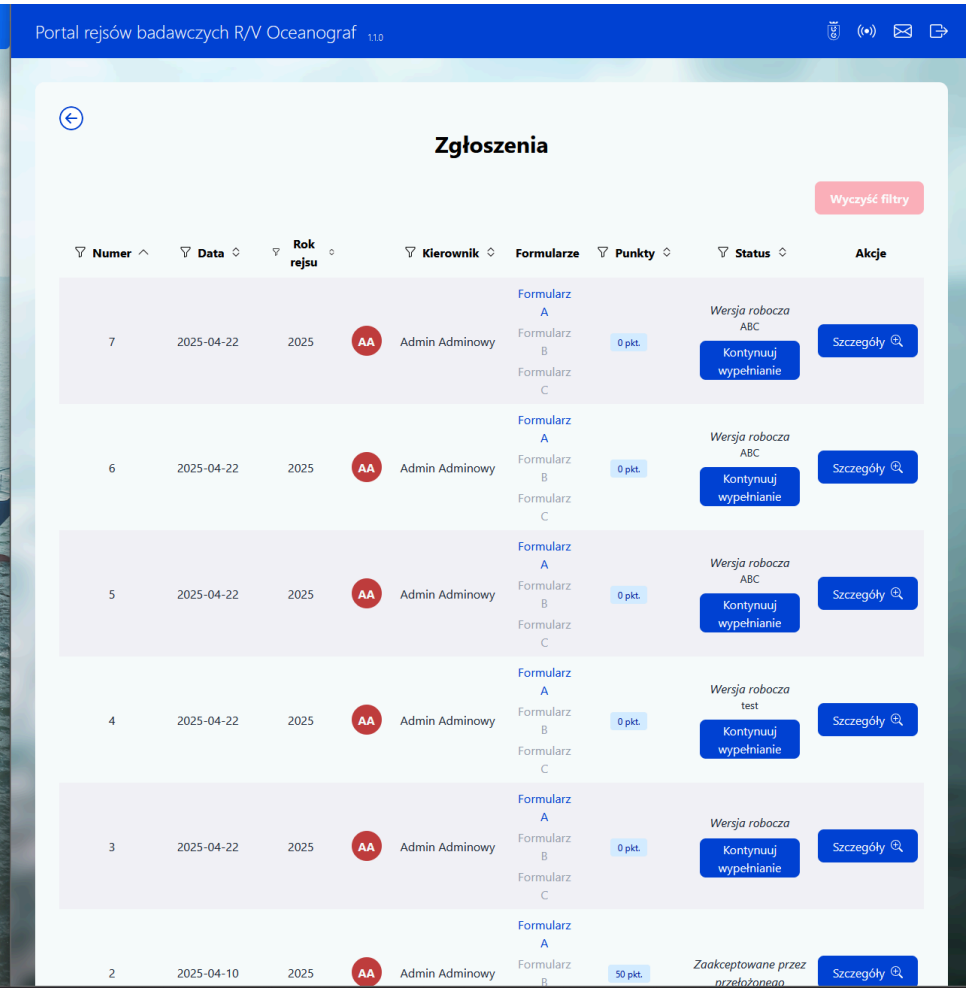
Porównajmy ponownie UX/UI przed i po





Porównajmy ponownie UX/UI przed i po

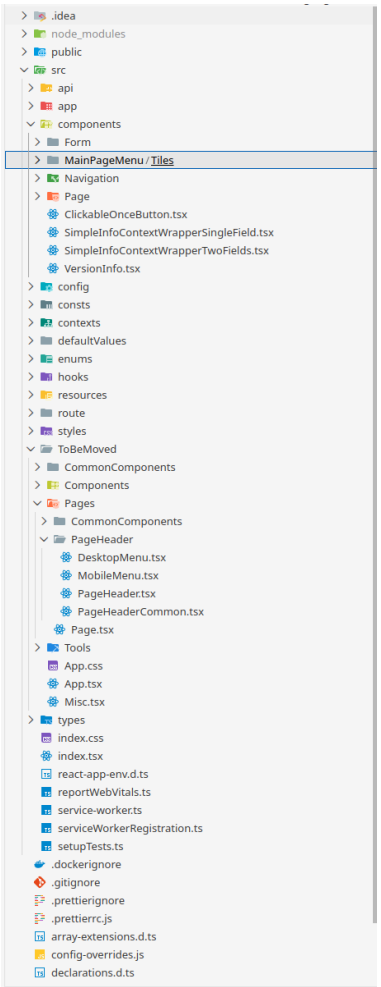




Dlaczego przepisaliśmy frontend

- Poprzednia wersja była mało wydajna, nieczytelna oraz miała wiele błędów UX/UI.
- Użytkownicy doświadczali losowych problemów takich jak wylogowania przez błędną rotację tokenów, utratę stanu aplikacji w trakcie nawigacji czy brak poprawnego wyświetlania błędów.
- Ograniczone filtrowanie/sortowanie oraz chaotyczny UX utrudniały codzienną pracę.
- Kod był niespójny, trudny do utrzymania i (co najgorsze) stylowany jednym plikiem SCSS nadpisującym Bootstrap (`-.-||`).

Kod przed refaktorem



Kod przed refaktorem

Explorer (Ctrl+Alt+L) tsx 1

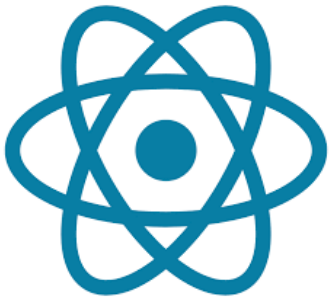
src > ToBeMoved > Components > Pages > FormPage > Inputs > PublicationsTable > PublicationsTable.tsx > ...

```
1   import { CategoryPicker, InformationColumn, MinisterialPointsField, YearField } from './PublicationsTableFields';
2   import { BottomMenuWithHistory, OrdinalNumber, RemoveRowButton } from '@app/pages/FormPage/Inputs/TableParts';
3   import { FieldProps } from '@app/pages/FormPage/Inputs/FormRadio';
4   import { useContext } from 'react';
5   import { FieldTableWrapper } from '@app/pages/FormPage/Wrappers/FieldTableWrapper';
6   import { FieldValues } from react-hook-form;
7   import { FormContext } from '@contexts/FormContext';
8   import { FieldContextWrapper } from '@app/pages/FormPage/Inputs/CrewTable/CrewTable';
9   import FieldWrapper from '@app/pages/FormPage/Inputs/FieldWrapper';
10
11  export const notEmptyArray = <T extends object>(value: FieldValues) => {
12      if (value.some((row: T) => {
13          return Object.values(row).some(field => {
14              return !field;
15              },
16          );
17      })) {
18          return 'Wypełnij wszystkie pola';
19      }
20  };
21
22  export type Publication = {
23      category: string,
24      doi: string,
25      authors: string,
26      title: string,
27      magazine: string,
28      year: string,
29      ministerialPoints: string
30  }
31
32  const publicationDefaultValues = [
33      {
34          category: 'subject',
35          doi: '',
36          authors: '',
37          title: '',
38          magazine: '',
39          year: '0',
40          ministerialPoints: '0',
41      },
42      {
43          category: 'postscript',
44          doi: '',
45          authors: '',
46          title: '',
47          magazine: '',
48          year: '0',
49          ministerialPoints: '0',
50      }];
51
52  export const publicationCategories = [
53      'subject',
54      'postscript',
55  ];
56
57  export const publicationCategoriesPL = [
58      'Temat',
59      'Dopisek',
60  ];
61
62
63  export const publicationOptions = publicationCategoriesPL.map((taskLabel, index) =>
64      ({ label: taskLabel, value: publicationDefaultValues[index] }));
65
66  const PublicationTableContent = () =>
67      {
68          () => (<OrdinalNumber label={'Publikacja'} />),
69          CategoryPicker,
70          InformationColumn,
71          YearField,
72          MinisterialPointsField,
73          RemoveRowButton,
74      };
75
76  type PublicationsTableProps = FieldProps &
77      { historicalPublications?: Publication[] }
78
79  const PublicationRowLabel = (row: Publication) =>
80      `DOI: ${row.doi}\n
81      Autorzy: ${row.authors}\n
82      Tytuł: ${row.title}\n
83      Czasopismo: ${row.magazine}\n
84      Rok wydania: ${row.year}\n
85      Punkty: ${row.ministerialPoints}`;
86
87
88  export const PublicationsTable = (props: PublicationsTableProps) => {
89
90      const formContext = useContext(FormContext);
91
92      const FilteredHistoricalPublications = (category: string) =>
93          props.historicalPublications?.filter((row) => row.category == category)
94              .map((row: Publication) =>
95                  ({ label: PublicationRowLabel(row), value: row })) ?? [];
96
97      const selectOptions = publicationCategories.map((publicationCategory, index) =>
98          ({
99              label: publicationCategoriesPL[index],
100             options: FilteredHistoricalPublications(publicationCategory),
101         }) ?? [];
102
103
104     const mdColWidths = [5, 15, 51, 10, 14, 5];
105     const mdColTitles = ['Lp.', 'Kategoria', 'Informacje', 'Rok wydania', 'Punkty ministerialne', ''];
106     const colTitle = 'Publikacje';
107     const bottomMenu =
108         <BottomMenuWithHistory newOptions={publicationOptions} historicalOptions={selectOptions} />;
109     const emptyText = 'Nie dodano żadnej publikacji';
110     const { Render } = FieldTableWrapper(colTitle, mdColWidths, mdColTitles, PublicationTableContent,
111         bottomMenu, emptyText, formContext!.getValues(props.fieldName));
112
113
114     const fieldProps = {
115         ...props,
116         defaultValue: [],
117         rules: {
```

Kod przed refaktorem

- Bardzo dużo powtórzeń kodu.
- Nadmierne wykorzystanie patternów i wzorców projektowych + React.
- Brak spójności w nazewnictwie i strukturze plików.
- Wiele komponentów w jednym pliku.
- Zatrważająco dużo `any` w TypeScript.

Wykorzystane technologie



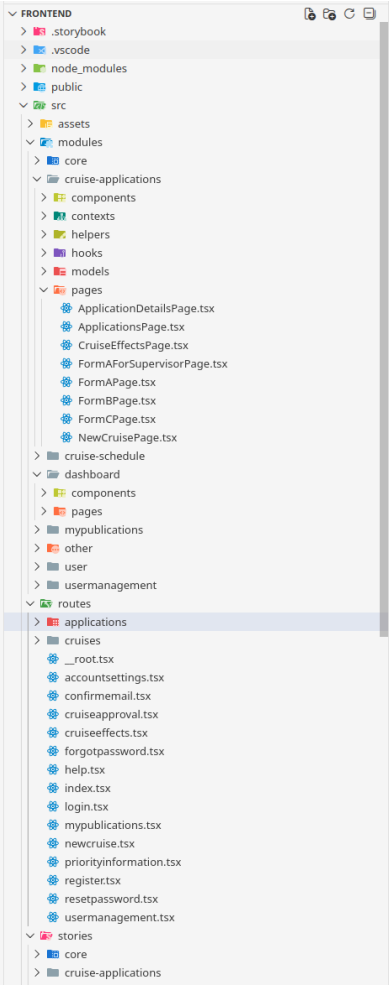
Uzasadnienie wyboru technologii

- React + TypeScript jako baza dzięki doświadczeniu zespołu i popularności ekosystemu.
- Vite zamiast create-react-app/react-app-rewired: szybszy build/dev, prostsza konfiguracja.
- Tailwind CSS dla izolowanych, komponentowych stylów; rezygnacja z ciężkiego SCSS nadpisującego Bootstrap.
- Storybook do podglądu i testów komponentów UI, wsparcie dla wdrożenia kolejnych zespołów.
- Motion - animacje i przejścia dla lepszego UX.

Uzasadnienie wyboru technologii

- TanStack Router: lepsza integracja z danymi, typowane ścieżki, wbudowane stany ładowania.
- TanStack Query zamiast czystego axios: cache odpowiedzi, retry idempotentnych żądań, praca offline.
- TanStack Forms: reaktywne formularze, walidacja na bazie zod, ochrona przed utratą danych.
- TanStack Table: generyczne filtrowanie/sortowanie, widok list na mobile, wyszukiwarka w tabelach.
- Własna biblioteka komponentów zamiast zewnętrznych UI kits dla pełnej spójności stylu.

Kod po refaktoringu



Kod po refaktoringu

src > modules > cruise-applications > components > application-details > ApplicationDetails.tsx > ApplicationDetails > acceptApplication

```
1  import { useQueryClient } from '@tanstack/react-query';
2  import toast from 'react-hot-toast';
3
4  import { ApplicationDetailsProvider } from '@/cruise-applications/contexts/ApplicationDetailsContext';
5  import {
6    useAcceptApplicationMutation,
7    useRejectApplicationMutation,
8  } from '@/cruise-applications/hooks/CruiseApplicationsApiHooks';
9  import { CruiseApplicationDto } from '@/cruise-applications/models/CruiseApplicationDto';
10 import { EvaluationDto } from '@/cruise-applications/models/EvaluationDto';
11
12 import { ApplicationDetailsActionsSection } from './ApplicationDetailsActionsSection';
13 import { ApplicationDetailsContractsSection } from './ApplicationDetailsContractsSection';
14 import { ApplicationDetailsEffectPointsSection } from './ApplicationDetailsEffectPointsSection';
15 import { ApplicationDetailsInformationSection } from './ApplicationDetailsInformationSection';
16 import { ApplicationDetailsMembersSection } from './ApplicationDetailsMembersSection';
17 import { ApplicationDetailsPublicationsSection } from  './ApplicationDetailsPublicationsSection';
18 import { ApplicationDetailsResearchTasksSection } from './ApplicationDetailsResearchTasksSection';
19 import { ApplicationDetailsSPUBTasksSection } from './ApplicationDetailsSPUBTasksSection';
20
21 type Props = {
22   application: CruiseApplicationDto;
23   evaluation: EvaluationDto;
24 };
25 export function ApplicationDetails({ application, evaluation }: Props) {
26   const queryClient = useQueryClient();
27
28   const acceptMutation = useAcceptApplicationMutation();
29   const rejectMutation = useRejectApplicationMutation();
30
31   function acceptApplication() {
32     acceptMutation.mutate(application.id, {
33       onSuccess: () => {
34         queryClient.invalidateQueries({ queryKey: ['cruiseApplications', application.id] });
35         toast.success('Formularz został zaakceptowany');
36       },
37       onError: (err) => {
38         console.error(err);
39         toast.error('Nie udało się zaakceptować formularza');
40       },
41     });
42   }
43
44   function rejectApplication() {
45     rejectMutation.mutate(application.id, {
46       onSuccess: () => {
47         queryClient.invalidateQueries({ queryKey: ['cruiseApplications', application.id] });
48         toast.success('Formularz został odrzucony');
49       },
50       onError: (err) => {
51         console.error(err);
52         toast.error('Nie udało się odrzucić formularza');
53       },
54     });
55   }
56
57   return (
58     <ApplicationDetailsProvider value={{ application, evaluation }}>
59       <ApplicationDetailsInformationSection />
60       <ApplicationDetailsResearchTasksSection />
61       <ApplicationDetailsEffectPointsSection />
62       <ApplicationDetailsContractsSection />
63       <ApplicationDetailsMembersSection />
64       <ApplicationDetailsPublicationsSection />
65       <ApplicationDetailsSPUBTasksSection />
66       <ApplicationDetailsActionsSection onAccept={acceptApplication} onReject={rejectApplication} />
67     </ApplicationDetailsProvider>
68   );
69 }
70
```

Kod po refaktoringu

src > modules > core > components > AppAccordion.tsx > AppAccordion

```
1   import ChevronDownIcon from 'bootstrap-icons/icons/chevron-down.svg?react';
2   import ChevronUpIcon from 'bootstrap-icons/icons/chevron-up.svg?react';
3   import { AnimatePresence, motion } from 'motion/react';
4   import React from 'react';
5
6   type Props = {
7     title: string;
8     children: React.ReactNode;
9     expandedByDefault?: true | undefined;
10  };
11
12  export function AppAccordion({ title, children, expandedByDefault = undefined }: Props) {
13    const [expanded, setExpanded] = React.useState<boolean>(!!expandedByDefault);
14
15    return (
16      <>
17        <h2 className="w-full">
18          <button
19            type="button"
20            className="w-full flex justify-between items-center cursor-pointer px-4 py-4 bg-black/2 rounded-xl"
21            onClick={() => setExpanded(!expanded)}
22          >
23            <span className="font-semibold text-lg">{title}</span>
24            <span>{expanded ? <ChevronUpIcon className="w-6 h-6" /> : <ChevronDownIcon className="w-6 h-6" />}</span>
25          </button>
26        </h2>
27        <AnimatePresence initial={false}>
28          {expanded && (
29            <motion.div
30              initial={{ opacity: 0, height: 0 }}
31              animate={{ opacity: 1, height: 'auto' }}
32              exit={{ opacity: 0, height: 0 }}
33              transition={{ ease: 'easeOut' }}
34              className="px-4"
35            >
36              {children}
37            </motion.div>
38          )}
39        </AnimatePresence>
40      </>
41    );
42  }
43
```

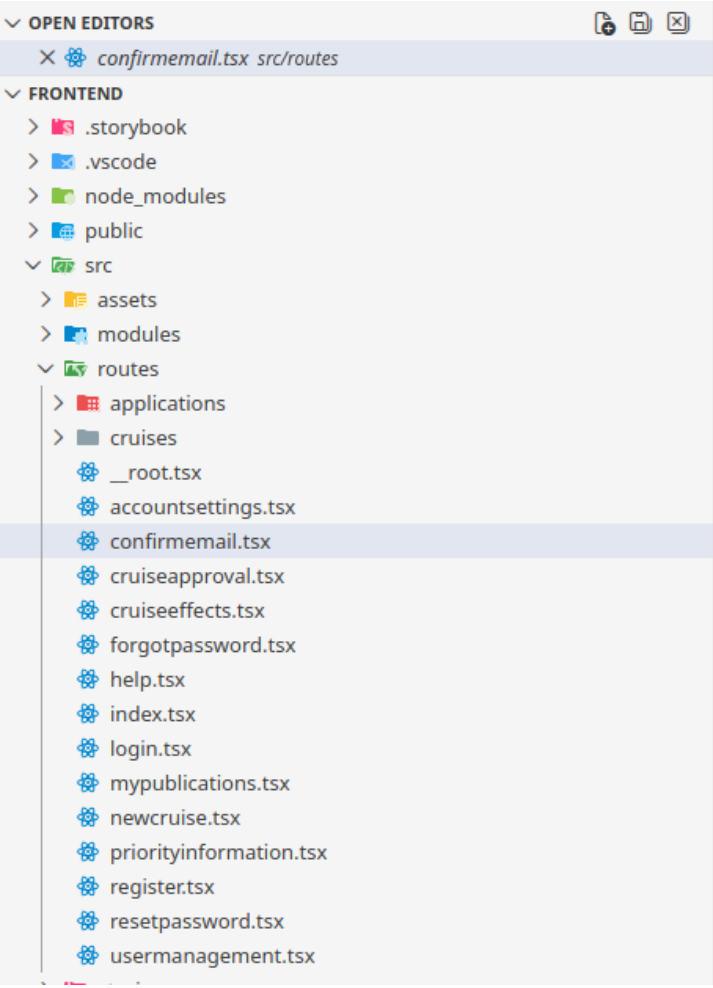
Kod po refaktoringu


```
type SupervisorFormQueryProps = {
  cruiseId: string;
  supervisorCode: string;
};
export function useFormAForSupervisorInitValuesQuery({ cruiseId, supervisorCode }: SupervisorFormQueryProps) {
  return useSuspenseQuery({
    queryKey: ['formAForSupervisorInitValues', cruiseId, supervisorCode],
    queryFn: async () => {
      return client.get(
        `/forms/InitValuesForSupervisor/A?cruiseApplicationId=${cruiseId}&supervisorCode=${supervisorCode}`
      );
    },
    select: (res) => res.data as FormAInitValuesDto,
  });
}

export function useFormAForSupervisorQuery({ cruiseId, supervisorCode }: SupervisorFormQueryProps) {
  return useSuspenseQuery({
    queryKey: ['formAForSupervisor', cruiseId, supervisorCode],
    queryFn: async () => {
      return client.get(`/api/CruiseApplications/${cruiseId}/formAForSupervisor?supervisorCode=${supervisorCode}`);
    },
    select: (res) => {
      const dto = res.data as FormADto;
      dto.note ??= '';
      return dto;
    },
  });
}

type SaveFormAProps = {
  form: FormADto;
  draft: boolean;
};
export function useSaveFormAMutation() {
  return useMutation({
    mutationFn: async ({ form, draft }: SaveFormAProps) => {
      return client.post(`/api/CruiseApplications?isDraft=${draft}`, form);
    },
  });
}
```

Kod po refaktoringu



src > routes > confirmemail.tsx > ...

```
1  import { createFileRoute } from '@tanstack/react-router';
2  import { z } from 'zod';
3
4  import { allowOnly } from '@/core/lib/guards';
5  import { ConfirmEmailPage } from '@/user/pages/ConfirmEmailPage';
6
7  export const Route = createFileRoute('/confirmemail')({
8    component: ConfirmEmailPage,
9    beforeLoad: allowOnly.unauthenticated(),
10   validateSearch: z.object({
11     userId: z.string().uuid().optional(),
12     code: z.string().optional(),
13   }),
14 });
15
```

Storybook

Storybook

Find components

STORIES

- core
  - components
    - AppAccordion
      - Docs
      - Expanded
      - Collapsed
    - AppActionsSection
    - AppAlert
    - AppAvatar
      - Docs
      - Default
      - Small
    - AppBadge
    - AppButton
    - AppLayout
    - AppLink
    - AppModal
    - AppPopover
    - AppTabs
    - calendar
    - inputs
    - layout
    - table
- cruise-applications
- cruise-schedule
- dashboard
- other

AppAvatar

Default
Small

SN

Show code

Name	Description	Default	Control
fullName*	string	-	Stanisław Nieradko
variant	union	'default'	default / small
className	string	-	Set string

STORIES

Default

JAS

Show code

Small

Wyzwania w trakcie

- Odświeżanie tokenu poza React Query (brak dostępu do kontekstu).
- Walidacja z dwiema konfiguracjami zod w locie okazała się kłopotliwa; błąd naprawiono przy przekazaniu projektu.
- Kalkulacja pozycji dropdownów dla dużych ekranów miała błąd matematyczny (naprawa po 3 dniach debugowania).
- Druk PDF (react-to-print): konieczność duplikacji uproszczonych komponentów formularzy wydłużyła prace.

Rezultaty po wdrożeniu

- Feature parity przy zachowaniu znanego stylu, ale z lepszym UX/UI i stabilnością.
- Wyższa wydajność potwierdzona Lighthouse: lepsze LCP (Largest Contentful Paint) oraz TBT (Total Blocking Time)
- Uporządkowany kod (komponentowe style, wyraźna struktura katalogów) ułatwia utrzymanie.
- Aplikacja dostępna u klienta od 20.11.2025; nowy zespół wdrożony i rozwija kolejne funkcje.

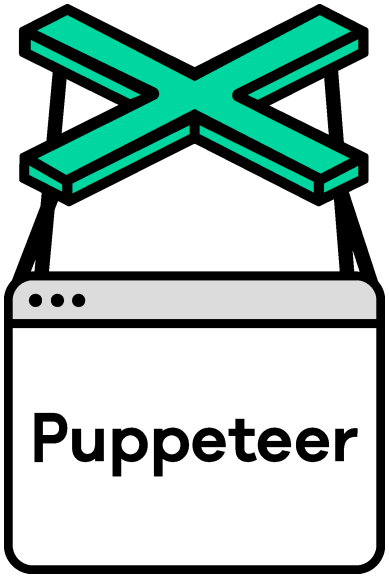
Porównanie narzędzi do testów automatycznych

Porównanie narzędzi do testów automatycznych

Najpopularniejsze rozwiązania stosowane w branży:

Selenium

Cypress



Puppeteer

Playwright

Selenium

Zalety:

- Szerokie wsparcie dla wielu przeglądarek, między innymi `Google Chrome`, `Firefox`, `Safari`, a nawet `Internet Explorer`

Wady:

- Brak oficjalnego wsparcia dla TypeScript
- Nieintuicyjne czekanie na oczekiwaną wartość

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

❌ Taki kod może w niektórych przypadkach nie zadziałać

Selenium

```
// Click the button
const button = await driver.findElement(By.id("myButton"));
await button.click();

// Check updated textbox value
const textbox = await driver.findElement(By.id("myTextbox"));

// Wait until checkbox becomes the wanted value
await driver.wait(async () => {
  const val = await textbox.getAttribute("value");
  return val === "Updated Value";
}, 5000);
const val = await textbox.getAttribute("value");
assert.strictEqual(value, "Updated Value");
```

Cypress

Zalety:

- Oficjalne wsparcie dla TypeScript
- Zaimplementowany domyślny, automatyczny sposób czekania na oczekiwaną wartość
- Udostępnia wiele narzędzi do inspekcji przebiegu testów

Wady:

- Duża część narzędzi jest dostępna tylko w dodatkowo płatnym `Cypress Cloud`
- W pełni oficjalnie wspierana jest tylko przeglądarka `Google Chrome`, z ograniczonym wsparciem dla przeglądarek `Firefox` i `Safari`

Cypress

```
// Click the button
cy.get('#myButton').click();

// Check updated textbox value
cy.get<HTMLInputElement>('#myTextbox').should('have.value', 'Updated Value');
```

✅ Taki kod zaczeka na oczekiwaną wartość przez ustalony z góry czas (np. domyślnie 30 s), nie trzeba samemu pisać kodu czekającego, jak to miało miejsce w Selenium

Puppeteer

Zalety:

- Oficjalne wsparcie dla TypeScript

Wady:

- Aktualnie kompatybilny tylko z `Google Chrome` i `Firefox`
- Podobnie jak w Selenium, nieintuicyjne czekanie na zmianę wartości

Puppeteer

```
// Click the button
await page.click("#myButton");

// Wait until textbox value becomes the expected value
await page.waitForFunction(
  () => (document.getElementById("myTextbox")
          as HTMLInputElement)?.value === "Updated Value",
  { timeout: 5000 }
);

// Get updated textbox value
const value = await page.$eval("#myTextbox",
  el => (el as HTMLInputElement).value);
assert.strictEqual(value, "Updated Value");
```

Playwright

Zalety:

- Kompatybilność ze wszystkimi współcześnie popularnymi przeglądarkami (`Google Chrome`, `Firefox` i `Safari`), jak i również z ich wersjami mobilnymi
- Oficjalne wsparcie dla TypeScript
- Domyślne czekanie na zmianę wartości na oczekiwaną
- Łatwa inspekcja przebiegu testów

Wady:

- Trochę gorsze narzędzia do inspekcji niż Cypress w płatnym wariancie

Playwright

```
// Click the button
await page.click('#myButton');

// Check updated textbox value
const textbox = page.locator('#myTextbox');
await expect(textbox).toHaveValue('Updated Value');
```

✅ Podobnie jak w Cypressie, tutaj również program automatycznie zaczeka przez ustalony czas

Podsumowanie

Po analizie różnych dostępnych rozwiązań, zdecydowaliśmy się na zastosowanie narzędzia **Playwright**.

Wybrane szczegóły implementacyjne w aplikacji

Frontend - Nowoczesny Stos Technologiczny

Aplikacja wykorzystuje najnowsze dostępne narzędzia w ekosystemie React.

- **React 19 RC**: Wykorzystanie funkcji biblioteki przed oficjalnym stabilnym wydaniem
- **Vite**: wydajny build tool
- **Ekosystem TanStack**:
  - `@tanstack/react-query`: Zarządzanie stanem serwera, caching
  - `@tanstack/react-table`: Headless UI do budowania skomplikowanych tabel
  - `@tanstack/react-form`: Zarządzanie stanem formularzy i walidacją
  - `@tanstack/react-router`: Type-safe routing
- **Tailwind CSS**: Najnowsza wersja silnika CSS

Backend - Czysta Architektura

Backend zaprojektowany zgodnie z zasadami Domain-Driven Design (DDD) i Clean Architecture.

- **Podział na warstwy**: `Domain` (Core), `Application` (Use Cases), `Infrastructure` (Implementation), `Web` (API)
- **CQRS**: Rozdzielenie operacji odczytu (Queries) od zapisu (Commands) przy użyciu **MediatR**
- **Separacja logiki**: Każda operacja biznesowa to osobny, izolowany Handler

Backend - Generator Kluczy (Custom Logic)

Generator identyfikatorów semantycznych (np. `2024/1`).

```
// Infrastructure/Services/YearBasedKeyGenerator.cs
public async Task<string> GenerateKey<T>(IRepository<T> repository, ...)
{
    var currentYear = DateTime.Now.Year.ToString(); // "2024"
    var entities = await repository.GetAll(cancellationToken);

    var maxOrdinal = entities
        .Where(e => e.Number.StartsWith(currentYear))
        .MaxBy(e => int.Parse(e.Number.Split('/')[1]))
        ?.Number.Split('/')[1] ?? "0";

    return $"{currentYear}/{int.Parse(maxOrdinal) + 1}";
}
```

Logika biznesowa przejścia stanu (cofnięcie do edycji).

```
// Application/.../RefillFormBHandler.cs
public async Task<Result> Handle(RefillFormBCommand request, ...)
{
    var app = await repo.GetById(request.Id);

    // Guard Clause: Sprawdzenie poprawności stanu
    if (app.Status != Status.Undertaken && app.Status != Status.FormBFilled)
        return Error.ForbiddenOperation("Nie można edytować.");

    app.Status = Status.FormBRequired; // Zmiana stanu
    await unitOfWork.Complete(); // Zapis
    return Result.Empty;
}
```

j - Tabela z Formularzem

Połączenie TanStack Table z React Form.

```
// frontend/.../FormAResearchAreaSection.tsx
{
  header: 'Rejon prowadzenia badań',
  cell: ({ row }) => (
    <form.Field
        name={`researchAreaDescriptions[${row.index}].differentName`}
        children={(field) => (
          <AppInput
             value={field.state.value}
             onChange={field.handleChange}
             errors={field.state.meta.errors} // Walidacja per wiersz
          />
        )}
    />
  )
}
```

Frontend - Zarządzanie Cyklem Życia (UI Logic)

Warunkowe renderowanie przycisków akcji w oparciu o status.

```
// frontend/.../CruiseDetailsPage.tsx
switch (cruiseQuery.data?.status) {
    case 'Nowy':
        return (
          <>
            <AppButton onClick={() => setEditMode(true)}>Edytuj</AppButton>
            <AppButton onClick={() => setIsConfirmModal(true)}>Zatwierdź</AppButton>
          </>
        );
    case 'Potwierdzony':
        return (
             <AppButton onClick={() => setIsRevertModal(true)}>Cofnij status</
AppButton>
        );
}
```

Backend - Bogaty Model Domenowy (Enum)

Wykorzystanie atrybutów do przechowywania przyjaznych nazw statusów bezpośrednio w kodzie.

```
// Domain/Common/Enums/CruiseApplicationStatus.cs
public enum CruiseApplicationStatus
{
    [StringValue("Wersja robocza")]
    Draft,

    [StringValue("Oczekujące na przełożonego")]
    WaitingForSupervisor,

    [StringValue("Zaakceptowane przez przełożonego")]
    AcceptedBySupervisor,

    // ...
}
```

Frontend - Custom Hook (Reużywalność)

Enkapsulacja logiki sprawdzania połączenia z API w prosty hook.

```
// modules/core/hooks/NetworkStatusApiHook.tsx
export function useNetworkStatusMutation({ setNetworkConnectionStatus }:
Props) {
  // Wywołanie prostego endpointu 'health'
  return useMutation({
    mutationFn: async () => client.get('/health'),
    onSuccess: () => setNetworkConnectionStatus(true),
    onError: () => setNetworkConnectionStatus(false),
    retry: false,
  });
}
```

Frontend - Type Safety (DTO)

Silne typowanie danych przesyłanych między formularzem a API.

```
// modules/cruise-schedule/models/CruiseFormDto.ts
export type CruiseFormDto = {
  startDate: string;
  endDate: string;
  managersTeam: {
      mainCruiseManagerId: string;
      mainDeputyManagerId: string;
  };
  cruiseApplicationsIds: string[];
  status?: string;
  shipUnavailable: boolean;
};
```

Transformacja danych domenowych na wydarzenia w kalendarzu.

```
// modules/cruise-schedule/components/CruiseCalendar.tsx
const events = React.useMemo(
  () =>
    cruises.map((cruise) => ({
      title: getTitle(cruise),
      start: new Date(cruise.startDate),
      end: new Date(cruise.endDate),
      link: `/cruises/${cruise.id}`,
      // Dynamiczne kolorowanie w zależności od typu
      color: cruise.shipUnavailable ? `#a10000` :
getHashColor(cruise.title),
    })),
  [cruises]
);
return <AppCalendar events={events} />;
```

Dziękujemy za uwagę!